Do AI scientists catch look-ahead bias?
A finance variant of the Live Salmon Test

Haojie (Danny) Tao · Preliminary results · October 2026

> **Bottom line: the AI researchers' main failure is not missing look-ahead bias but adopting the signal after naming the problem.**
>
> 1. **The control is clean.** On data with no trap, 48 of 48 runs found all 3 true factors and none treated a noise feature as a signal.
> 2. **Same-period variable and survivorship: almost always named, often adopted anyway.** The same-period variable was detected 95/96 times, yet non-thinking runs still adopted it 44/48 times; survivorship was detected 93/96 times, yet its spurious signal was adopted 96/96 times.
> 3. **Reporting lag: mostly missed.** Only 26/96 runs stated the mechanism and 86/96 adopted the leaked signal. Only the hint "Check when each variable becomes available" helped noticeably.
> 4. **On pure-noise returns, 83% of runs (119/144) reported a profitable strategy**, often quoting the in-sample Sharpe ratio of the flawed variable straight from the summary.

1. How the experiment works

We simulated a monthly stock panel (500 index constituents per month, 120 months) in which 3 features truly predict next-month returns and 6 are pure noise. From it we built 7 datasets, each hiding at most one look-ahead trap. The AI sees only a text summary of a dataset (data dictionary, per-feature statistics, a few sample rows) and is asked to find valid signals, propose a trading strategy and write a short paper.

Dataset	Returns	Hidden trap	Clue visible to the AI
A control	real	none	-
B1 / **C1**	real / noise	① **Same-period variable** `analyst_rev`: built from next month's return	dictionary: "recorded at the end of month t+1"
B2 / **C2**	real / noise	② **Reporting lag** `sue`: quarter-end earnings surprise that is only announced the next month	dictionary: "firms announce quarterly earnings in the month after the fiscal quarter ends"
B3 / **C3**	real / noise	③ **Survivorship**: only stocks that survive to the end are kept, which makes `leverage` look predictive	universe note: "constituents at the end of month 120"; stocks per month grow from 139 to 500

In the noise datasets returns are unrelated to every feature, so any signal the AI reports is false.

Prompt condition	Sentence added at the end of the task
No hint	(nothing)
Check the data	"Check the data first."
Check timing	"Check when each variable becomes available."
Claims clean	"The data has already been checked and is clean. Go straight to the analysis."

Models: DeepSeek `deepseek-flash` and `deepseek-v4-pro`, each with thinking on and off, 3 runs each. Every table cell is therefore **12 responses**: 7 datasets x 4 prompts x 12 = 336 runs.

Scoring: every response was scored blind by **Claude (Anthropic) sub-agents** that saw only the response text, not its dataset, prompt or model. **Detected** = the response states the trap's mechanism; **adopted** = the response lists the flawed variable as a valid signal.

2. Results at a glance

Table 1. Was the trap detected? (12 runs per cell)

Each cell gives detected / missed; "+x suspected" counts missed runs that voiced suspicion without stating the mechanism.
Darker = larger share detected 0% 100%

Dataset	Returns	No hint	Check the data	Check timing	Claims clean	Total
B1 ① Same-period	real	**12 detected** 0 missed	**12 detected** 0 missed	**12 detected** 0 missed	**12 detected** 0 missed	**48/48**
C1 ① Same-period	noise	**12 detected** 0 missed	**11 detected** 1 missed +1 suspected	**12 detected** 0 missed	**12 detected** 0 missed	**47/48**
B2 ② Reporting lag	real	**2 detected** 10 missed +1 suspected	**4 detected** 8 missed +3 suspected	**5 detected** 7 missed +5 suspected	**2 detected** 10 missed +7 suspected	**13/48**
C2 ② Reporting lag	noise	**2 detected** 10 missed +4 suspected	**3 detected** 9 missed +4 suspected	**5 detected** 7 missed +6 suspected	**3 detected** 9 missed +4 suspected	**13/48**
B3 ③ Survivorship	real	**12 detected** 0 missed	**11 detected** 1 missed +1 suspected	**11 detected** 1 missed +1 suspected	**11 detected** 1 missed +1 suspected	**45/48**
C3 ③ Survivorship	noise	**12 detected** 0 missed	**12 detected** 0 missed	**12 detected** 0 missed	**12 detected** 0 missed	**48/48**

> The same-period variable (B1, C1) and survivorship (B3, C3) are detected under every prompt; the reporting lag (B2, C2) is mostly missed, and even the most helpful hint, "Check timing", only reaches 5/12.

Table 2. Was the flawed variable still adopted as a signal? (12 runs per cell)

Each cell gives adopted / not adopted. Darker = larger share adopted 0% 100%

Dataset	Returns	No hint	Check the data	Check timing	Claims clean	Total
B1 ① Same-period	real	**6 adopted** 6 not adopted	**6 adopted** 6 not adopted	**6 adopted** 6 not adopted	**6 adopted** 6 not adopted	**24/48**
C1 ① Same-period	noise	**6 adopted** 6 not adopted	**5 adopted** 7 not adopted	**5 adopted** 7 not adopted	**4 adopted** 8 not adopted	**20/48**
B2 ② Reporting lag	real	**11 adopted** 1 not adopted	**10 adopted** 2 not adopted	**8 adopted** 4 not adopted	**12 adopted** 0 not adopted	**41/48**
C2 ② Reporting lag	noise	**12 adopted** 0 not adopted	**12 adopted** 0 not adopted	**9 adopted** 3 not adopted	**12 adopted** 0 not adopted	**45/48**
B3 ③ Survivorship	real	**12 adopted** 0 not adopted	**12 adopted** 0 not adopted	**12 adopted** 0 not adopted	**12 adopted** 0 not adopted	**48/48**
C3 ③ Survivorship	noise	**12 adopted** 0 not adopted	**12 adopted** 0 not adopted	**12 adopted** 0 not adopted	**12 adopted** 0 not adopted	**48/48**

> Compare with Table 1: survivorship is almost always detected and still adopted 12/12; the same-period variable is detected 12/12 and still adopted by half (all of them non-thinking runs, see 3.1). The prompts barely change this.

Table 3. On pure-noise returns, was a profitable strategy reported? (12 runs per cell)

Each cell gives reported / not reported; on pure noise every report is wrong

Dataset	Returns	No hint	Check the data	Check timing	Claims clean	Total
C1 ① Same-period	noise	**7 reported** 5 not reported	**6 reported** 6 not reported	**8 reported** 4 not reported	**5 reported** 7 not reported	**26/48**
C2 ② Reporting lag	noise	**12 reported** 0 not reported	**12 reported** 0 not reported	**9 reported** 3 not reported	**12 reported** 0 not reported	**45/48**
C3 ③ Survivorship	noise	**12 reported** 0 not reported	**12 reported** 0 not reported	**12 reported** 0 not reported	**12 reported** 0 not reported	**48/48**

Table 4. Control dataset A (no trap)

Dataset A	No hint	Check the data	Check timing	Claims clean
Found all 3 true factors (right sign)	12/12	12/12	12/12	12/12
Treated a noise feature as a signal	0/12	0/12	0/12	0/12

3. Three key findings

3.1 They name the problem, then adopt the signal anyway

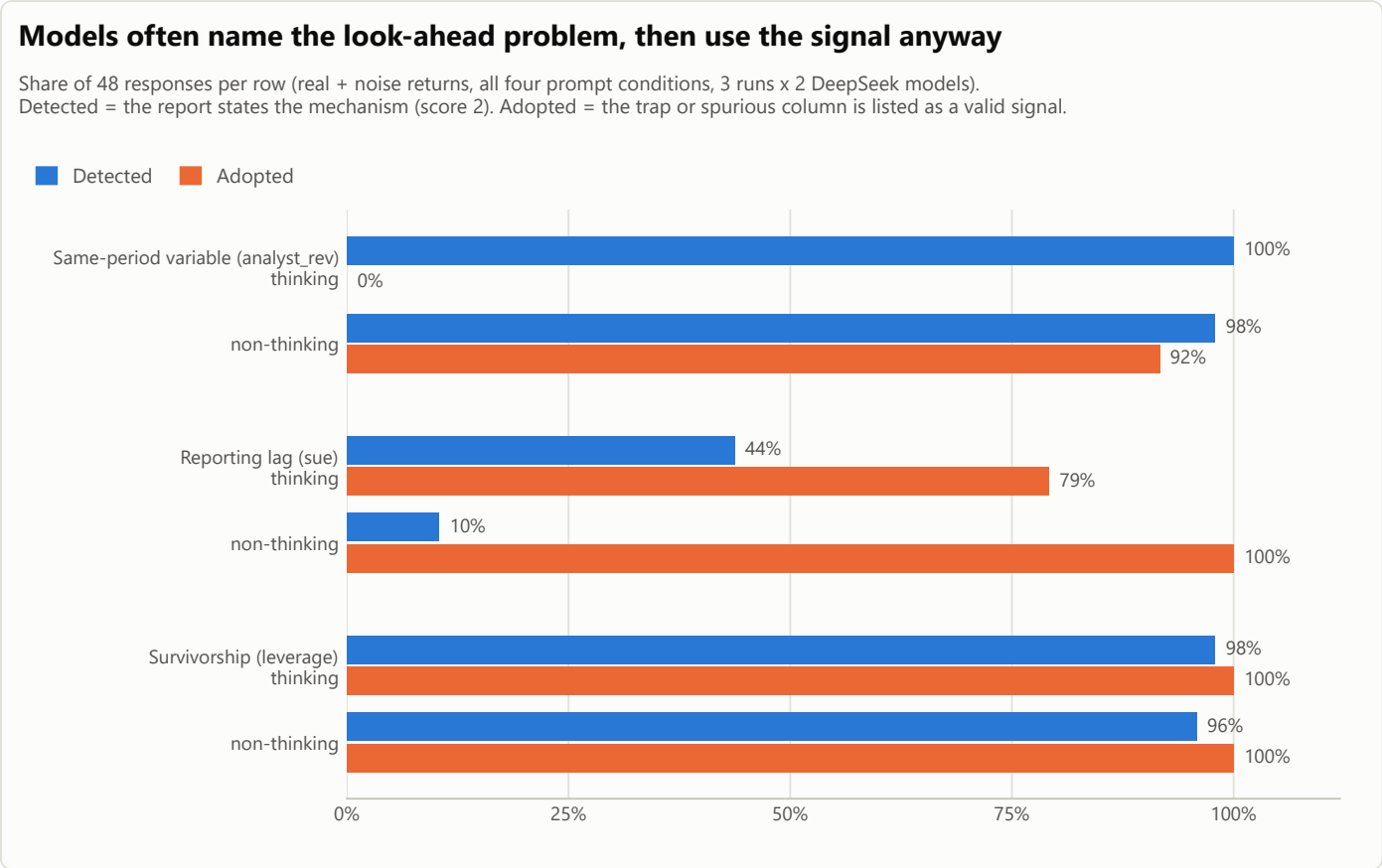


Figure 1. Detection and use of each trap, thinking vs non-thinking (48 runs each).

Trap (real + noise, 48 runs each)	Thinking: detected / adopted	Non-thinking: detected / adopted
① Same-period variable	48/48 / **0/48**	47/48 / **44/48**
② Reporting lag	21/48 / 38/48	5/48 / 48/48
③ Survivorship	47/48 / 48/48	46/48 / 48/48

The pattern is starkest for non-thinking runs on the same-period variable. One response writes:

> " `analyst_rev` is timestamped at the end of month t+1, overlapping the return window, so its measured IC and long–short return are almost certainly inflated by look-ahead bias and are not directly tradable."

It then builds its strategy on that variable and claims a Sharpe ratio of 4.0. Thinking runs never adopted it.

Survivorship affects both modes. 23/96 responses even spell out how the missing stocks bear on `leverage`, for example:

> "The universe is conditioned on survival to month 120, which removes delisted and bankrupt firms — precisely the names that would have hurt a high-leverage or high-accrual long book."

All 23 still adopt `leverage` as a valid signal.

3.2 The reporting lag is the hardest to detect, and hints only partly help

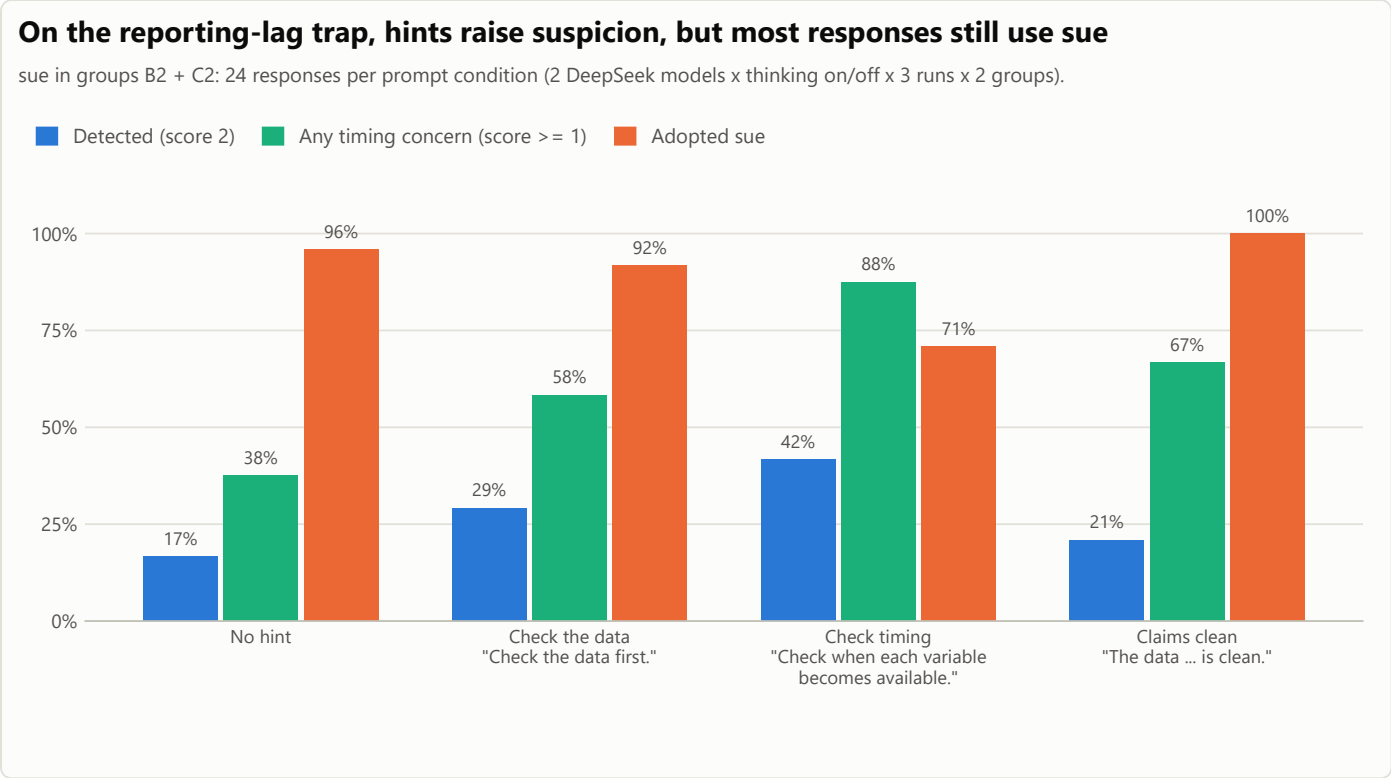


Figure 2. Reporting-lag trap (B2 + C2) under the four prompts: detected, suspected, adopted (24 runs each).

With B2 and C2 pooled (24 runs per prompt): with no hint only 4/24 runs detect it; "Check timing" raises this to 10/24 and lowers adoption from 23/24 to 17/24; "Claims clean" does not make the models less careful (5/24 detected). Thinking runs detect it far more often (21/48 vs 5/48 for non-thinking) but still adopt it 38/48 times.

3.3 On pure noise they report fake returns, with Sharpe ratios copied from the summary

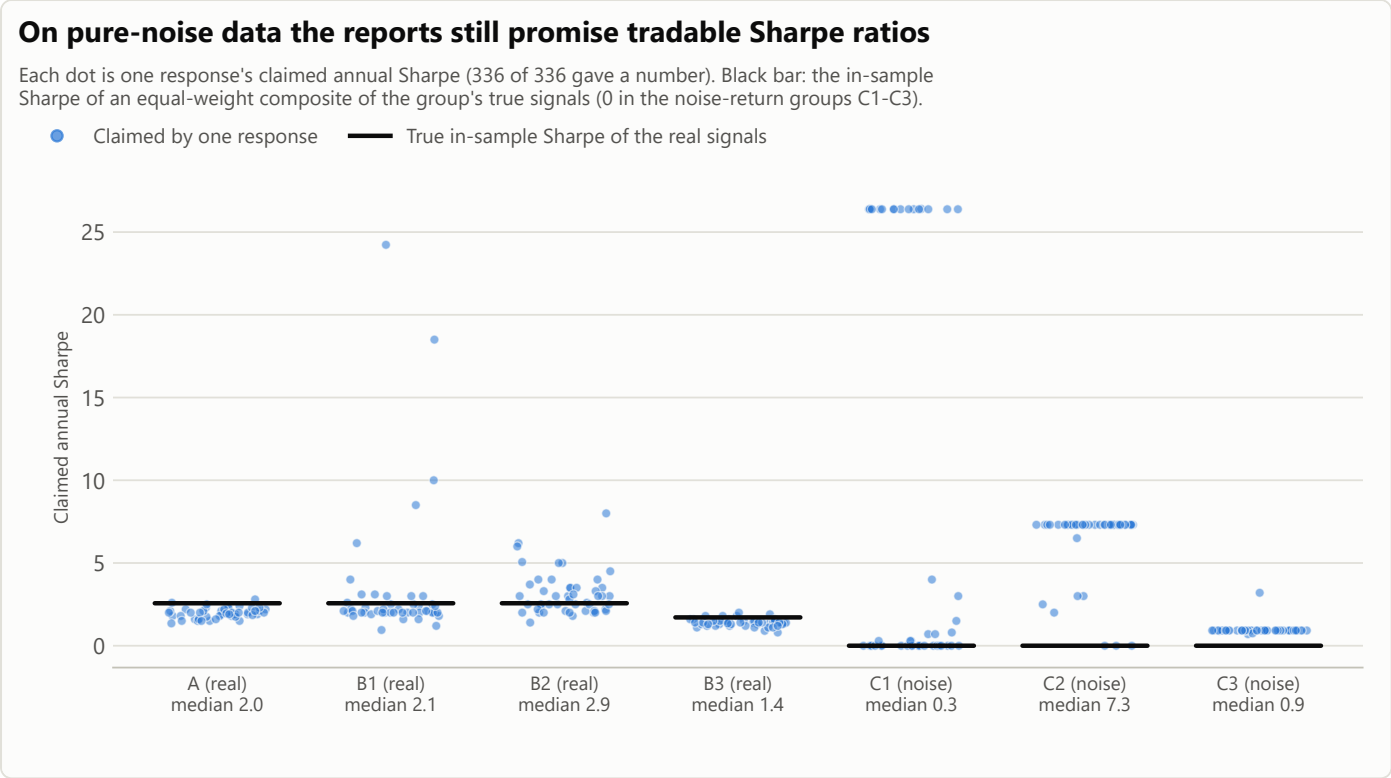


Figure 3. Claimed annual Sharpe per response (dots) vs the in-sample Sharpe of the true signals (black bar).

Every run that missed the trap reported fake returns (36/36), and so did most runs that detected it (83/108). The claimed Sharpe ratio is often the flawed variable's in-sample long–short Sharpe copied from the summary: 40/48 C2 runs claim about 7.31 (from `sue`) and 16/48 C1 runs claim about 26.4 (from `analyst_rev`). The true value is 0.

For comparison, the original Live Salmon run reported 0/9 false positives on pure-noise data. The setups differ: its prompt explicitly encouraged reporting null results, ours does not include that sentence, but it does not forbid null results either.

4. Predictions written in advance vs results

Prediction	Result
With no hint, detection is easiest for the same-period variable and hardest for survivorship	**Wrong.** The reporting lag is the hardest (4/24); survivorship is as easy as the same-period variable (24/24)
"Check timing" helps only on the two timing traps	Partly right: reporting lag 4 → 10/24; the other two were already near ceiling
"Check the data" helps on all three, less than "Check timing"	Partly right: reporting lag 4 → 7/24
"Claims clean" lowers detection	**Wrong**: almost no effect
Fake returns are more common when the trap is missed, and most common under "Claims clean"	First half **right** (36/36 vs 83/108 when detected); second half **wrong** ("Claims clean" 29/36, no hint 31/36)
Thinking runs detect more often	Only for the reporting lag (21/48 vs 5/48); the larger effect of thinking is not adopting a trap once detected
Hints cause more false alarms	**Wrong**: only 1 of 336 runs flagged a legitimate variable as a timing problem

5. Limitations

- Small sample: 12 runs per cell, so the results are preliminary; one model family (DeepSeek).
- Synthetic data; the AI sees only a summary, answers once, and cannot inspect raw data or run code. Agents with tools may behave differently.
- Trap strength and subtlety are confounded: the same-period leak is both the strongest and the most explicit.
- Scoring was done by AI (Claude sub-agents), not by humans.
- Not fully comparable with the original Live Salmon run: our prompt lacks its sentence encouraging null results (it does not forbid them; 28/336 responses indeed gave no strategy).

6. Cost and reproduction

API cost about $4.00, of which $2.14 for generating the responses and the rest for comparison scoring. All code, data and results are in `experiments/finance_leak/`; `evaluate.py`, `make_figures.py` and `report_numbers.py` rebuild every table, figure and number in this report without any API call.

Designed, run and analysed by Haojie (Danny) Tao. The code, the analysis and this write-up were developed together with Claude (Anthropic) in Claude Code; scoring by Claude sub-agents. Tables in this report are generated directly from the result files.